

HelixCode Zero-Bluff Master Implementation Plan

Document Purpose

This is the authoritative implementation roadmap to transform HelixCode from its current state (with verified bluff areas) into a **fully complete, zero-bluff project** that exceeds all Tier 1 CLI agent capabilities. Every phase includes specific file references, implementation details, and anti-bluff verification criteria.

Date: 2026-04-30 **Target:** Zero-bluff, production-ready, Tier 1+ capability **Governance:** Based on HelixAgent CONSTITUTION.md CONST-001 through CONST-036 + CONST-035 anti-bluff mandate

PHASE 0: FOUNDATION REPAIR (Week 1)

P0-001: Fix go.mod - Add All Advertised Dependencies

File: `HelixCode/go.mod` (replace root go.mod) **Current State:** Only 3 dependencies (uuid, errors, yaml) **Required State:** Full dependency manifest

```
module dev.helix.code

go 1.24.0

toolchain go1.24.9

require (
    // HTTP Framework
    github.com/gin-gonic/gin v1.11.0

    // Authentication
    github.com/golang-jwt/jwt/v4 v4.5.2
    golang.org/x/crypto v0.36.0

    // Database
    github.com/jackc/pgx/v5 v5.7.4

    // Redis
    github.com/redis/go-redis/v9 v9.7.3

    // Configuration
    github.com/spf13/viper v1.21.0

    // CLI
    github.com/spf13/cobra v1.8.0

    // Testing
    github.com/stretchr/testify v1.11.1

    // UI
    github.com/rivo/tview v0.42.0
    fyne.io/fyne/v2 v2.7.0

    // Browser Automation
    github.com/chromedp/chromedp v0.14.2
```

```

// Web Scraping
github.com/PuerkitoBio/goquery v1.10.3

// Tree-sitter
github.com/smacker/go-tree-sitter v0.0.0-20240625050157-a31a98a7c127

// Utilities
github.com/google/uuid v1.6.0
github.com/pkg/errors v0.9.1
gopkg.in/yaml.v2 v2.4.0

// Observability
github.com/prometheus/client_golang v1.22.0
github.com/sirupsen/logrus v1.9.3

// SSE for MCP
github.com/r3labs/sse/v2 v2.10.0
)

```

Anti-Bluff Verification:

- ☐ `go mod tidy` completes without errors
- ☐ `go build ./...` compiles all packages
- ☐ `go list -m all` shows all dependencies resolved
- ☐ CI script verifies no missing imports

P0-002: Create Missing docker-entrypoint.sh

File: `docker/docker-entrypoint.sh` (new file) **Current State:** Referenced in Dockerfile but doesn't exist

```

#!/bin/sh
set -e

# HelixCode Container Entrypoint
# Anti-bluff: This script MUST actually start services, not just echo

# Validate required environment variables
if [ -z "$HELIX_AUTH_JWT_SECRET" ]; then
    echo "ERROR: HELIX_AUTH_JWT_SECRET is required"
    exit 1
fi

if [ -z "$HELIX_DATABASE_PASSWORD" ]; then
    echo "WARNING: HELIX_DATABASE_PASSWORD not set - database will be unavailable"
fi

# Wait for PostgreSQL if configured
if [ -n "$HELIX_DATABASE_URL" ]; then
    echo "Waiting for PostgreSQL..."
    until pg_isready -d "$HELIX_DATABASE_URL" 2>/dev/null; do
        sleep 1
    done
    echo "PostgreSQL is ready"

```

```

fi

# Wait for Redis if configured
if [ -n "$HELIX_REDIS_URL" ]; then
    echo "Waiting for Redis..."
    until redis-cli -u "$HELIX_REDIS_URL" ping 2>/dev/null | grep -q PONG; do
        sleep 1
    done
    echo "Redis is ready"
fi

# Run database migrations if available
if [ -f "./scripts/migrate.sh" ]; then
    echo "Running database migrations..."
    ./scripts/migrate.sh || true
fi

# Start the appropriate service based on HELIX_SERVICE_TYPE
case "${HELIX_SERVICE_TYPE:-server}" in
    server)
        echo "Starting HelixCode server..."
        exec ./server
        ;;
    cli)
        echo "Starting HelixCode CLI..."
        exec ./cli "$@"
        ;;
    worker)
        echo "Starting HelixCode worker..."
        exec ./worker "$@"
        ;;
    *)
        echo "Unknown service type: $HELIX_SERVICE_TYPE"
        exit 1
        ;;
esac

```

Anti-Bluff Verification:

- ☐ Container builds successfully: `docker build -t helixcode:test .`
- ☐ Container starts without "file not found" errors
- ☐ Container actually waits for dependencies (not just echoes)
- ☐ Health endpoint responds after container starts

P0-003: Add Root-Level Governance Files

Files:

- `CONSTITUTION.md` (new)
- `CLAUDE.md` (new)
- `AGENTS.md` (update existing)

These files MUST propagate to ALL submodules. See separate governance document for full text.

PHASE 1: CORE LLM INTEGRATION (Weeks 2-3)

P1-001: Implement Real LLM Provider Interface

File: [internal/llm/provider.go](#) (verify/update) **Current State:** Interface may exist but implementations are stubs

Required Provider Interface (already partially defined):

```
type Provider interface {
    Generate(ctx context.Context, req *GenerateRequest) (*GenerateResponse, error)
    GenerateStream(ctx context.Context, req *GenerateRequest) (<-chan *GenerateChunk, error)
    GetCapabilities() *Capabilities
    GetModels() ([]Model, error) // CHANGED: must return real models
    ValidateConfig(config map[string]interface{}) error
    HealthCheck(ctx context.Context) error // NEW: for anti-bluff verification
}
```

P1-002: Implement Ollama Provider (Local - Priority 1)

File: [internal/llm/providers/ollama/ollama.go](#) (new or fix)

```
package ollama

import (
    "bytes"
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "time"
)

const defaultBaseURL = "http://localhost:11434"

type OllamaProvider struct {
    baseURL    string
    httpClient *http.Client
}

func New(baseURL string) *OllamaProvider {
    if baseURL == "" {
        baseURL = defaultBaseURL
    }
    return &OllamaProvider{
        baseURL: baseURL,
        httpClient: &http.Client{Timeout: 120 * time.Second},
    }
}

func (p *OllamaProvider) Generate(ctx context.Context, req *llm.GenerateRequest) (*llm.GenerateResponse, error) {
    // ANTI-BLUFF: This MUST make a real HTTP call to Ollama API
```

```
// NO simulation. NO echoing prompt. NO canned responses.

payload := map[string]interface{}{
    "model": req.Model,
    "prompt": req.Prompt,
    "stream": false,
    "options": map[string]interface{}{
        "temperature": req.Temperature,
        "num_predict": req.MaxTokens,
    },
}

body, _ := json.Marshal(payload)
httpReq, _ := http.NewRequestWithContext(ctx, "POST", p.baseURL+"/api/generate",
bytes.NewReader(body))
httpReq.Header.Set("Content-Type", "application/json")

resp, err := p.httpClient.Do(httpReq)
if err != nil {
    return nil, fmt.Errorf("ollama generate failed: %w", err)
}
defer resp.Body.Close()

if resp.StatusCode != http.StatusOK {
    return nil, fmt.Errorf("ollama returned %d", resp.StatusCode)
}

var result struct {
    Response string `json:"response"`
    Done      bool   `json:"done"`
}
if err := json.NewDecoder(resp.Body).Decode(&result); err != nil {
    return nil, fmt.Errorf("failed to decode ollama response: %w", err)
}

return &llm.GenerateResponse{
    Text:      result.Response,
    Model:     req.Model,
    Provider:  "ollama",
    Completed: result.Done,
}, nil
}

func (p *OllamaProvider) GetModels() ([]llm.Model, error) {
    // ANTI-BLUFF: Must query Ollama's /api/tags endpoint
    // Return REAL available models, not hardcoded list
    resp, err := p.httpClient.Get(p.baseURL + "/api/tags")
    if err != nil {
        return nil, fmt.Errorf("failed to list ollama models: %w", err)
    }
    defer resp.Body.Close()

    var result struct {
        Models []struct {
            Name      string `json:"name"`
            Size      int64  `json:"size"`
            Digest    string `json:"digest"`
        } `json:"models"`
    }

```

```

    }
    if err := json.NewDecoder(resp.Body).Decode(&result); err != nil {
        return nil, err
    }

    models := make([]llm.Model, len(result.Models))
    for i, m := range result.Models {
        models[i] = llm.Model{
            ID:      m.Name,
            Name:    m.Name,
            Provider: "ollama",
            // Infer context size from model name patterns
        }
    }
    return models, nil
}

func (p *OllamaProvider) HealthCheck(ctx context.Context) error {
    // ANTI-BLUFF: Real health check, not just "return nil"
    req, _ := http.NewRequestWithContext(ctx, "GET", p.baseURL+"/api/tags", nil)
    resp, err := p.httpClient.Do(req)
    if err != nil {
        return fmt.Errorf("ollama health check failed: %w", err)
    }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK {
        return fmt.Errorf("ollama health check returned %d", resp.StatusCode)
    }
    return nil
}

```

P1-003: Implement OpenAI Provider (Cloud - Priority 2)

File: `internal/llm/providers/openai/openai.go` (new)

Similar pattern with real HTTP calls to <https://api.openai.com/v1/chat/completions>.

P1-004: Implement Provider Manager with Fallback

File: `internal/llm/manager.go` (verify/fix)

```

type ProviderManager struct {
    providers map[string]Provider
    fallbackChain []string // Ordered list of provider names for fallback
    circuitBreakers map[string]*CircuitBreaker // NEW: per-provider circuit breakers
}

func (m *ProviderManager) Generate(ctx context.Context, req *GenerateRequest) (*GenerateResponse, error) {
    // ANTI-BLUFF: Must try REAL providers, not return simulated response
    for _, providerName := range m.fallbackChain {
        provider := m.providers[providerName]
        if provider == nil {
            continue
        }
    }
}

```

```

    cb := m.circuitBreakers[providerName]
    if cb != nil && cb.State() == CircuitOpen {
        continue // Skip unhealthy providers
    }

    resp, err := provider.Generate(ctx, req)
    if err == nil {
        if cb != nil {
            cb.RecordSuccess()
        }
        return resp, nil
    }

    if cb != nil {
        cb.RecordFailure()
    }
}

return nil, fmt.Errorf("all providers failed")
}

```

P1-005: Fix CLI handleGenerate to Use Real Providers

File: cmd/cli/main.go lines 190-214 (CRITICAL FIX)

Replace the simulated generation with real provider calls:

```

func (c *CLI) handleGenerate(ctx context.Context, prompt, model string, maxTokens int,
temperature float64, stream bool) error {
    // ANTI-BLUFF: This MUST use the real LLM provider
    if c.llmProvider == nil {
        // Initialize provider from config or default to local
        provider, err := c.initializeProvider(model)
        if err != nil {
            return fmt.Errorf("failed to initialize provider: %w", err)
        }
        c.llmProvider = provider
    }

    req := &llm.GenerateRequest{
        Prompt:    prompt,
        Model:     model,
        MaxTokens: maxTokens,
        Temperature: temperature,
    }

    if stream {
        return c.handleGenerateStream(ctx, req)
    }

    resp, err := c.llmProvider.Generate(ctx, req)
    if err != nil {
        return fmt.Errorf("generation failed: %w", err)
    }

    fmt.Println(resp.Text)
}

```

```
    return nil
}
```

P1-006: Implement Circuit Breaker Pattern

File: `internal/llm/circuit_breaker.go` (new - port from HelixAgent)

Port the verified real implementation from HelixAgent:

```
type CircuitBreaker struct {
    state      CircuitState
    failureCount int
    successCount int
    threshold  int
    timeout     time.Duration
    lastFailure time.Time
    mu          sync.RWMutex
}

type CircuitState int

const (
    CircuitClosed CircuitState = iota
    CircuitOpen
    CircuitHalfOpen
)
```

Anti-Bluff Verification for Phase 1:

- ☐ `go test ./internal/llm/...` passes
- ☐ Integration test calls real Ollama instance (or mock server in test)
- ☐ `./bin/helixcode --prompt "What is 2+2?"` returns actual AI-generated text
- ☐ `./bin/helixcode --list-models` returns dynamic models from running provider
- ☐ Health check verifies provider is actually responding
- ☐ Challenge script generates a real project using LLM calls
- ☐ NO "simulated" or "for now" comments in production LLM code

PHASE 2: TOOLS & EDITOR (Weeks 4-5)

P2-001: Implement Real Filesystem Tools

Files: `internal/tools/filesystem/*.go` **Current State:** Interfaces may exist, need real implementations

Required tools:

- `fs_read` - Read file contents with path validation
- `fs_write` - Write file with atomic operations
- `fs_edit` - Edit file with backup
- `glob` - Pattern matching
- `grep` - Content search

Anti-Bluff Requirements:

- ☐ Tools actually read/write files on disk
- ☐ Path validation prevents directory traversal
- ☐ Backups created before edits
- ☐ Integration tests use real temp files

P2-002: Implement Real Shell Tool with Sandboxing

File: `internal/tools/shell/shell.go`

```
func (t *ShellTool) Execute(ctx context.Context, params map[string]interface{})
(map[string]interface{}, error) {
    // ANTI-BLUFF: This MUST execute real shell commands
    // But with strict security boundaries

    command := params["command"].(string)

    // Validate against blocklist
    if err := t.validateCommand(command); err != nil {
        return nil, fmt.Errorf("command blocked: %w", err)
    }

    // Execute with timeout and resource limits
    ctx, cancel := context.WithTimeout(ctx, t.maxExecutionTime)
    defer cancel()

    cmd := exec.CommandContext(ctx, "sh", "-c", command)
    cmd.Dir = t.workingDirectory

    // ANTI-BLUFF: Actually run the command, don't simulate
    // output, err := cmd.CombinedOutput()

    return map[string]interface{}{
        "stdout":    string(output),
        "exit_code": cmd.ProcessState.ExitCode(),
        "success":   err == nil,
    }, nil
}
```

P2-003: Implement Git Tool

File: `internal/tools/git/git.go`

Required capabilities:

- Status, diff, add, commit
- Branch listing, checkout
- Blame for attribution
- Smart commit message generation

P2-004: Implement Editor with Multi-Format Support

File: `internal/editor/editor.go`

Required formats:

- Diff format (unified diff)
- Whole file replacement
- Search/Replace with regex
- Line-based edits

Anti-Bluff Verification for Phase 2:

- ☐ `fs_write` actually creates files
- ☐ `shell` actually runs commands and returns real output
- ☐ `git_status` returns real git status
- ☐ Editor formats apply actual changes to files
- ☐ Challenge: Edit a file and verify the change persists

PHASE 3: WORKER & DISTRIBUTED COMPUTING (Weeks 6-7)

P3-001: Verify SSH Worker Implementation

Files: `internal/worker/ssh.go`, `internal/worker/pool.go` **Current State:** Unknown - need to verify if real or simulated

If simulated, implement:

- Real SSH connection management using golang.org/x/crypto/ssh
- Health checks with actual TCP connection attempts
- Task distribution with result collection
- Auto-installation via SSH exec

P3-002: Implement Task Checkpointing

File: `internal/task/checkpoint.go`

```
type Checkpoint struct {
    TaskID      uuid.UUID
    Sequence    int
    State       map[string]interface{}
    CreatedAt   time.Time
}

func (m *TaskManager) CreateCheckpoint(ctx context.Context, taskID uuid.UUID, state map[string]interface{}) error {
    // ANTI-BLUFF: Must persist to real database
    checkpoint := &Checkpoint{
        TaskID:      taskID,
        Sequence:    m.getNextSequence(taskID),
        State:       state,
        CreatedAt:   time.Now(),
    }
    return m.db.SaveCheckpoint(ctx, checkpoint)
}
```

P3-003: Implement Real Task Distribution

File: `internal/task/distributor.go`

Tasks must be:

1. Assigned to real workers based on capability matching
2. Monitored for progress (not just marked "running")
3. Results collected from actual worker execution
4. Failover to other workers on failure

Anti-Bluff Verification for Phase 3:

- ☐ Worker pool connects to real SSH endpoints
- ☐ Task execution happens on remote workers
- ☐ Checkpoints are stored in PostgreSQL
- ☐ Task failures trigger retry on different workers
- ☐ Challenge: Distribute a build task across 2 workers

PHASE 4: WORKFLOW & SESSION (Weeks 8-9)

P4-001: Implement Workflow Engine

File: [internal/workflow/engine.go](#)

```
type WorkflowEngine struct {
    actions map[string]Action
    store   WorkflowStore
}

func (e *WorkflowEngine) Execute(ctx context.Context, workflow *Workflow)
(*WorkflowResult, error) {
    // ANTI-BLUFF: Must execute real steps, not just mark complete
    // Dependency resolution
    // Step execution with real action calls
    // Result collection
    // Rollback on failure
}
```

P4-002: Implement Session Management with Redis

File: [internal/session/manager.go](#)

Sessions must:

- Store context in real Redis (not memory)
- Handle TTL and expiration
- Support multi-session tracking
- Persist mentions, search results, history

P4-003: Implement Real Project Lifecycle

File: [internal/project/manager.go](#)

Projects must:

- Be stored in real database

- Support lifecycle transitions (planning -> building -> testing -> deploying)
- Track associated tasks, sessions, workers
- Generate actual project artifacts

Anti-Bluff Verification for Phase 4:

- ☐ Workflow executes all steps in order
 - ☐ Session data persists across restarts (via Redis)
 - ☐ Project state changes are saved to database
 - ☐ Challenge: Create project, run workflow, verify artifacts
-

PHASE 5: MCP, MEMORY & NOTIFICATIONS (Weeks 10-11)

P5-001: Full MCP Protocol Implementation

File: [internal/mcp/server.go](#)

Must support:

- stdio transport
- SSE transport
- Tool registration and execution
- Resource management
- Real bidirectional communication

P5-002: Real Memory Provider Integration

File: [internal/memory/providers/*.go](#)

For each advertised provider (Mem0, Zep, ChromaDB, etc.):

- Implement real API client
- Implement health check
- Test with real service (or skip if unavailable)

P5-003: Real Notification Dispatch

File: [internal/notification/channels/*.go](#)

For each channel:

- Slack: Real webhook POST
- Discord: Real webhook POST
- Email: Real SMTP connection
- Telegram: Real bot API call

Anti-Bluff Verification for Phase 5:

- ☐ MCP server accepts real connections
 - ☐ Memory providers store and retrieve actual data
 - ☐ Notifications arrive at real endpoints
 - ☐ Challenge: Send notification, verify receipt
-

PHASE 6: TESTING & CHALLENGES (Weeks 12-13)

P6-001: Anti-Bluff Test Framework

File: `tests/anti_bluff_framework.go` (new)

```
package tests

// AntiBluffVerifier ensures tests validate REAL behavior
// Usage: Every integration/E2E test must call at least one verification

type AntiBluffVerifier struct {
    t *testing.T
}

func (v *AntiBluffVerifier) AssertRealHTTPCall(resp *http.Response, err error) {
    // FAIL if response is simulated
    if err == nil && resp.StatusCode == 200 {
        body, _ := io.ReadAll(resp.Body)
        if strings.Contains(string(body), "simulated") ||
strings.Contains(string(body), "This is a simulated") {
            v.t.Fatalf("ANTI-BLUFF VIOLATION: Response contains simulated content")
        }
    }
}

func (v *AntiBluffVerifier) AssertRealFileChange(path string, expectedContent string)
{
    // FAIL if file wasn't actually modified
    content, err := os.ReadFile(path)
    if err != nil {
        v.t.Fatalf("ANTI-BLUFF VIOLATION: File not modified: %v", err)
    }
    if !strings.Contains(string(content), expectedContent) {
        v.t.Fatalf("ANTI-BLUFF VIOLATION: Expected content not found in file")
    }
}

func (v *AntiBluffVerifier) AssertRealProcessExecution(cmd string) {
    // FAIL if command execution is simulated
    // Check that actual system calls were made
}
```

P6-002: Honest Challenge Framework

File: `tests/e2e/challenges/framework.go`

Challenge requirements (per CONST-035):

1. Challenge MUST exercise the actual feature end-to-end
2. Challenge MUST fail if the feature is simulated/stubbed
3. Challenge MUST verify output quality (not just existence)
4. Challenge MUST run against real infrastructure
5. Challenge wrapper MUST correctly propagate failure

Example challenge structure:

```
#!/bin/bash
# Challenge: llm_generation_001
# Validates that LLM generation produces real (non-simulated) responses

set -euo pipefail

LOG="/tmp/challenge_llm_$$log"
trap 'rm -f "$LOG"' EXIT

# Start server (if not running)
./bin/helixcode --daemon &
SERVER_PID=$!
trap 'kill $SERVER_PID 2>/dev/null; rm -f "$LOG"' EXIT
sleep 2

# Make real generation request
RESPONSE=$(curl -s -X POST http://localhost:8080/api/v1/llm/generate \
  -H "Content-Type: application/json" \
  -d '{"prompt":"What is the capital of France?","model":"llama-3-8b"}')

# ANTI-BLUFF: Verify response is not simulated
echo "$RESPONSE" >> "$LOG"

if echo "$RESPONSE" | grep -qi "simulated"; then
  echo "|FAILED| Challenge llm_generation_001: Response is simulated" | tee -a
"$LOG"
  exit 1
fi

if echo "$RESPONSE" | grep -qi "This is a simulated"; then
  echo "|FAILED| Challenge llm_generation_001: Response contains simulation marker"
| tee -a "$LOG"
  exit 1
fi

# ANTI-BLUFF: Verify response actually answers the question
if ! echo "$RESPONSE" | grep -qi "paris"; then
  echo "|FAILED| Challenge llm_generation_001: Response doesn't contain expected
answer" | tee -a "$LOG"
  exit 1
fi

echo "|PASSED| Challenge llm_generation_001: Real LLM generation verified"
```

P6-003: Master Challenge Runner

File: tests/e2e/challenges/run_all_challenges.sh

Must:

- Run ALL challenges
- Use robust failure detection (grep "|FAILED|", not exit codes alone)
- Generate JSON report with per-challenge results
- Fail if ANY challenge fails (no silent swallowing)
- Limit resource usage per Constitution (GOMAXPROCS=2, nice -n 19)

Anti-Bluff Verification for Phase 6:

- ☐ `run_all_challenges.sh` fails when BLUFF-001 is present
 - ☐ Test framework detects simulated responses
 - ☐ Wrapper correctly reports failures
 - ☐ Challenge validates actual end-user workflow
 - ☐ 100% of P0 features have challenges
 - ☐ 80% of P1 features have challenges
-

PHASE 7: DOCUMENTATION & DEPLOYMENT (Week 14)

P7-001: Update All Documentation

Files: README.md, docs/*.md

Every document MUST:

- Reflect actual implemented features (not aspirational)
- Include working code examples
- Be verified by challenge that executes examples

P7-002: Create Deployment Guide

File: docs/DEPLOYMENT.md

Must include:

- Docker Compose with real health checks
- Kubernetes manifests (port from HelixAgent)
- Environment variable reference
- Troubleshooting with real diagnostic commands

P7-003: Submodule Governance Propagation

Action: Create/update Constitution.md in each submodule

For each submodule in `.gitmodules`:

1. Check if Constitution.md exists
2. If not, create symlink or copy from parent
3. Verify AGENTS.md references anti-bluff rules

Anti-Bluff Verification for Phase 7:

- ☐ README examples work when copy-pasted
 - ☐ Docker Compose starts all services
 - ☐ All submodules have governance files
 - ☐ Documentation challenge passes
-

SUCCESS CRITERIA

HelixCode achieves zero-bluff status when ALL of the following are true:

1. ☐ `./bin/helixcode --prompt "What is 2+2?"` returns a real AI-generated answer (not simulated)

2.

`./bin/helixcode --list-models` returns dynamic models from running providers
3.

`./bin/helixcode --command "echo hello"` actually runs the command and returns "hello"
4.

`go build ./...` compiles ALL packages without errors
5.

`docker-compose up` starts all services, health checks pass
6.

`make test` runs unit tests with mocks only where allowed
7.

`make integration-test` runs against real PostgreSQL + Redis
8.

`./tests/e2e/challenges/run_all_challenges.sh` passes ALL challenges
9.

NO "simulated", "for now", "TODO", "placeholder" text in production code
10.

Constitution.md, CLAUDE.md, AGENTS.md exist at root and in all submodules
11.

All features advertised in README are exercised by at least one challenge
12.

HelixCode exceeds Aider's capability in: MCP support, distributed workers, memory integration

APPENDIX: Reference Implementations to Port from HelixAgent

HelixAgent File	HelixCode Target	Description
<code>internal/llm/circuit_breaker.go</code>	<code>internal/llm/circuit_breaker.go</code>	Production circuit breaker
<code>internal/llm/ensemble.go</code>	<code>internal/llm/ensemble.go</code>	Parallel provider execution
<code>internal/llm/providers/*/provider.go</code>	<code>internal/llm/providers/*/</code>	Real provider implementations
<code>internal/tools/handler.go</code>	<code>internal/tools/</code>	1000+ line tool handler
<code>internal/services/boot_manager.go</code>	<code>internal/server/boot.go</code>	Service boot orchestration
<code>challenges/scripts/run_all_challenges.sh</code>	<code>tests/e2e/challenges/run_all.sh</code>	Master challenge runner
<code>challenges/scripts/challenge_framework.sh</code>	<code>tests/e2e/challenges/framework.sh</code>	Challenge framework
<code>CONSTITUTION.md</code>	<code>CONSTITUTION.md</code>	36-rule governance
<code>CLAUDE.md</code>	<code>CLAUDE.md</code>	70KB agent manual
<code>docker-compose.yml</code>	<code>docker-compose.yml</code>	Production orchestration
<code>k8s/</code>	<code>k8s/</code>	Kubernetes manifests
<code>internal/skills/</code>	<code>internal/skills/</code>	Skills system

This plan is a living document. Every change to HelixCode MUST update this plan and verify against the anti-bluff criteria defined herein.